Assignment (Module II): Core Java

Task 1: University – Department – Professor (One-to-Many Relationship)
Problem
In real-world university systems, a **Department** manages several **Professors**, but each professor belongs to exactly one department. Instead of keeping all details inside a single class, we use separate entities for better modularity and scalability. This represents a **One-to-Many relationship** (one department → many professors).
Class Specifications:
• Department class:
o Private attributes: deptName (String), hodName (String), professors (List<Professor>)
o Constructors: default, parameterized (deptName, hodName)
o Getters/Setters for all fields
o Method: addProfessor(Professor p)
o Override toString() to return department + HOD details + professor list
• Professor class:
o Private attributes: name (String), employeeId (String), specialization (String)
o Constructors: default, parameterized (name, employeeId, specialization)
o Getters/Setters
o Override toString() to return professor details

Solution :

```
import java.util.*;

class Professor {
    private String name, employeeId, specialization;
    Professor() {}
    Professor(String n, String id, String s){ name=n; employeeId=id;
specialization=s; }

    String getName(){ return name; } void setName(String v){ name=v; }
    String getEmployeeId(){ return employeeId; } void setEmployeeId(String
v){ employeeId=v; }
    String getSpecialization(){ return specialization; } void
setSpecialization(String v){ specialization=v; }
```

```
    public String toString(){
        return "Name: "+name+", ID: "+employeeId+", Specialization:
"+specialization;
    }
}

class Department {
    private String deptName, hodName;
    private List<Professor> professors = new ArrayList<>();

    Department() {}
    Department(String d, String h){ deptName=d; hodName=h; }

    String getDeptName(){ return deptName; } void setDeptName(String v){
deptName=v; }
    String getHodName(){ return hodName; } void setHodName(String v){
hodName=v; }
    List<Professor> getProfessors(){ return professors; }
    void setProfessors(List<Professor> v){ professors=v; }

    void addProfessor(Professor p){ professors.add(p); }

    public String toString(){
        String s="Department: "+deptName+"\nHOD:
"+hodName+"\nProfessors:";
        for(Professor p:professors) s+="\n"+p;
        return s;
    }
}

public class UniversityDepartment {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] d=sc.nextLine().split(",",2);
        Department dep=new Department(d[0].trim(),d[1].trim());
```

```
        int n=Integer.parseInt(sc.nextLine().trim());
        while(n-->0){
            String[] p=sc.nextLine().split(",",3);
            dep.addProfessor(new
Professor(p[0].trim(),p[1].trim(),p[2].trim()));
        }
        System.out.println(dep);
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss>  & 'C:\Program Files\Java\jdk-24\bin\java.exe'
-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\workspaceStorage\62444c16c4ef0ccbf91112585e6
d7c32\bin' 'UniversityDepartment'
Computer Science,Dr. Mehta
2
Arjun,P101,AI
Neha,P102,ML
Department: Computer Science
HOD: Dr. Mehta
Professors:
Name: Arjun, ID: P101, Specialization: AI
Name: Neha, ID: P102, Specialization: ML
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 2: Flight Booking System (Abstraction + Inheritance)
Problem
Airline systems have **different types of flights** — domestic and international. While both share some common attributes (flight number, airline, fare), the **fare calculation rules differ**. Using **abstraction**, we can define a generic Flight and implement details in subclasses.
Class Specifications:
• Abstract Flight class:
o Private attributes: flightNumber (String), airline (String), fare (double)
o Constructor (flightNumber, airline, fare)
o Abstract method: calculateFare()
o Override toString() → flight details with final fare
• DomesticFlight class extends Flight:
o Fare = base fare + 10% tax
• InternationalFlight class extends Flight:
o Fare = base fare + 25% tax

Solution :

```
import java.util.*;

abstract class Flight {
    private String no, airline;
    private double fare;

    Flight(String n,String a,double f){ no=n; airline=a; fare=f; }
    double getFare(){ return fare; }
    abstract double calculateFare();

    public String toString(){
        return "Flight No: "+no+" Airline: "+airline+" Fare:
"+calculateFare();
    }
}

class DomesticFlight extends Flight {
    DomesticFlight(String n,String a,double f){ super(n,a,f); }
    double calculateFare(){ return getFare()*1.10; }
}

class InternationalFlight extends Flight {
    InternationalFlight(String n,String a,double f){ super(n,a,f); }
    double calculateFare(){ return getFare()*1.25; }
}
```

```
public class FlightBooking {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        for(int i=0;i<2;i++){
            String[] x=sc.nextLine().split(",",4);
            Flight f=x[0].trim().equalsIgnoreCase("Domestic")
                ? new
DomesticFlight(x[1].trim(),x[2].trim(),Double.parseDouble(x[3].trim()))
                : new
InternationalFlight(x[1].trim(),x[2].trim(),Double.parseDouble(x[3].trim()
));
            System.out.println(f);
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss> & 'C:\Program Files\Java\jdk-24\bi
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\workspaceStorag
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'FlightBooking'
Domestic,AI202,Air India,5000
International,QF101,Qantas,20000
Flight No: AI202 Airline: Air India Fare: 5500.0
Flight No: QF101 Airline: Qantas Fare: 25000.0
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 3: Employee Payroll System (Constructor Chaining + Overriding)
Problem
A company maintains **different types of employees**. A normal employee only has basic salary, while managers earn an additional bonus. This requires **inheritance, constructor chaining, and method overriding** for salary calculation.
Class Specifications:
• Employee class:
o Attributes: name, id, basicSalary
o Constructors: default, (name, id, basicSalary)
o Method: calculateSalary() → returns basicSalary
o Override toString() → employee details + salary
• Manager class extends Employee:
o Extra field: bonus
o Constructor calls super + initializes bonus
o Override calculateSalary() → salary = basicSalary + bonus

Solution :

```
import java.util.*;

class Employee {
    private String name,id;
    private double basicSalary;

    Employee() {}
    Employee(String n,String i,double s){ name=n; id=i; basicSalary=s; }

    String getName(){ return name; }
    String getId(){ return id; }
    double getBasicSalary(){ return basicSalary; }

    double calculateSalary(){ return basicSalary; }

    public String toString(){
        return "Employee "+name+" ("+id+") Salary: "+calculateSalary();
    }
}

class Manager extends Employee {
    private double bonus;

    Manager(String n,String i,double s,double b){ super(n,i,s); bonus=b; }

    double calculateSalary(){ return getBasicSalary()+bonus; }

    public String toString(){
```

```
        return "Manager "+getName()+" ("+getId()+") Salary:
"+calculateSalary();
    }
}

public class EmployeePayroll {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        for(int i=0;i<2;i++){
            String[] x=sc.nextLine().split(",");
            Employee e=x[0].trim().equalsIgnoreCase("Manager")
                ? new
Manager(x[1].trim(),x[2].trim(),Double.parseDouble(x[3].trim()),Double.par
seDouble(x[4].trim()))
                : new
Employee(x[1].trim(),x[2].trim(),Double.parseDouble(x[3].trim()));
            System.out.println(e);
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss>  & 'C:\Program Files\Java\jdk-24\bin\java.exe
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\workspaceStorage\62444c16
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'EmployeePayroll'
Employee,Ravi,E101,30000
Manager,Seema,M202,40000,5000
Employee Ravi (E101) Salary: 30000.0
Manager Seema (M202) Salary: 45000.0
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 4: Online Shopping Cart (Aggregation)
Problem
In e-commerce, an order contains multiple products. Instead of storing everything in one class, we model Order and Product. This represents aggregation because Order contains Products but both exist independently.
Class Specifications:
• Product class:
o Fields: productName, price, quantity
o Constructors + Getters/Setters
o toString() → productName x qty = total
• Order class:
o Fields: orderId, products (List<Product>)
o Method: calculateTotal()
o Override toString() to print order summary

Solution :

```
import java.util.*;

class Product {
    private String productName;
    private double price;
    private int quantity;

    Product() {}
    Product(String n,double p,int q){ productName=n; price=p; quantity=q;
}

    String getProductName(){ return productName; } void
setProductName(String v){ productName=v; }
    double getPrice(){ return price; } void setPrice(double v){ price=v; }
    int getQuantity(){ return quantity; } void setQuantity(int v){
quantity=v; }

    double total(){ return price*quantity; }
    public String toString(){ return productName+" x"+quantity+" =
"+(long)total(); }
}

class Order {
    private String orderId;
    private List<Product> products=new ArrayList<>();

    Order(String id){ orderId=id; }
    void addProduct(Product p){ products.add(p); }
```

```
    double calculateTotal(){
        double t=0;
        for(Product p:products) t+=p.total();
        return t;
    }

    public String toString(){
        String s="Order ID: "+orderId+"\nProducts:";
        for(Product p:products) s+="\n"+p;
        return s+"\nTotal: "+(long)calculateTotal();
    }
}

public class OnlineShoppingCart {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        Order o=new Order(sc.nextLine().trim());
        int n=Integer.parseInt(sc.nextLine().trim());

        while(n-->0){
            String[] x=sc.nextLine().split(",",3);
            o.addProduct(new
Product(x[0].trim(),Double.parseDouble(x[1].trim()),Integer.parseInt(x[2].
trim())));
        }
        System.out.println(o);
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss> & 'C:\Program Files\Java\jdk-24\bin\java.
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\workspaceStorage\62444
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'OnlineShoppingCart'
ORD101
3
Laptop,50000,1
Mouse,500,2
Keyboard,1500,1
Order ID: ORD101
Products:
Laptop x1 = 50000
Mouse x2 = 1000
Keyboard x1 = 1500
Total: 52500
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 5: Hospital Management (Multi-level Inheritance)
Problem
A hospital tracks different roles. A Person can be a Doctor, and a doctor can specialize as a Surgeon. This models a **multi-level inheritance** hierarchy.
Class Specifications:
• Person: name, age
• Doctor extends Person: specialization
• Surgeon extends Doctor: surgeryType
• Constructors + toString() chain

Solution :

```
import java.util.*;

class Person {
    private String name;
    private int age;
    Person(String n,int a){ name=n; age=a; }
    public String toString(){ return "Name: "+name+"\nAge: "+age; }
}

class Doctor extends Person {
    private String specialization;
    Doctor(String n,int a,String s){ super(n,a); specialization=s; }
    public String toString(){ return super.toString()+"\nSpecialization:
"+specialization; }
}

class Surgeon extends Doctor {
    private String surgeryType;
    Surgeon(String n,int a,String s,String t){ super(n,a,s);
surgeryType=t; }
    public String toString(){ return super.toString()+"\nSurgery Type:
"+surgeryType; }
}

public class HospitalManagement {
    public static void main(String[] args){
```

```
        Scanner sc=new Scanner(System.in);
        String[] x=sc.nextLine().split(",",4);
        System.out.println(new
Surgeon(x[0].trim(),Integer.parseInt(x[1].trim()),x[2].trim(),x[3].trim())
);
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss>  & 'C:\Program Files\Java\jdk-24\bin\
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\workspaceStorage\
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'HospitalManagement'
John,40,Cardiology,Heart Surgery
Name: John
Age: 40
Specialization: Cardiology
Surgery Type: Heart Surgery
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 6: Library – Book – Author (One-to-One Relationship)
Problem
Each book has a single author. We separate author details into another class for better modularity. This forms a **one-to-one relationship**.
Class Specifications:
• Author class: name, email, gender
• Book class: title, price, author
• toString() in Book prints Author details via composition

Solution :

```
import java.util.*;

class Author {
    private String name,email;
    private char gender;

    Author(String n,String e,char g){ name=n; email=e; gender=g; }

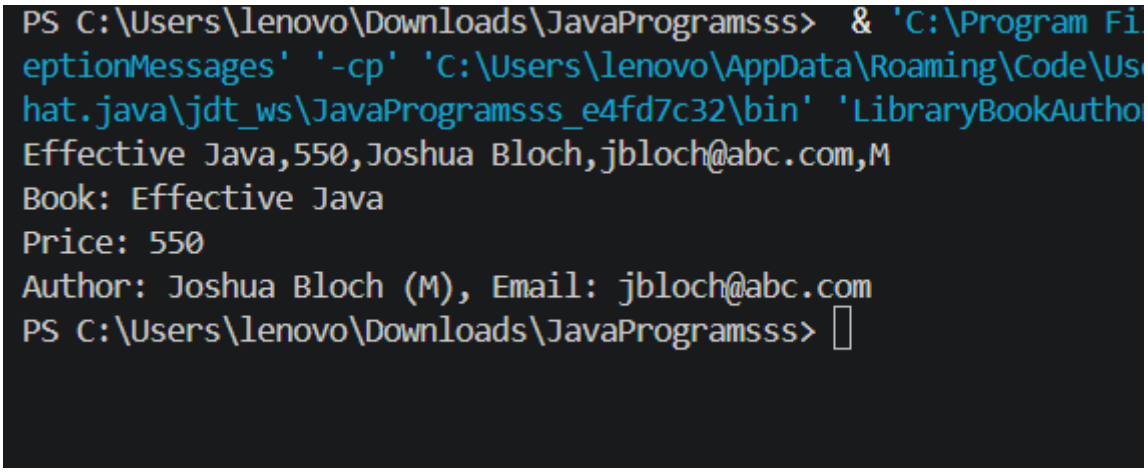
    public String toString(){
        return name+" ("+gender+"), Email: "+email;
    }
}

class Book {
    private String title;
    private double price;
    private Author author;

    Book(String t,double p,Author a){ title=t; price=p; author=a; }

    public String toString(){
        return "Book: "+title+"\nPrice: "+(long)price+"\nAuthor: "+author;
    }
}

public class LibraryBookAuthor {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] x=sc.nextLine().split(",",5);
        Author a=new
Author(x[2].trim(),x[3].trim(),x[4].trim().charAt(0));
        System.out.println(new
Book(x[0].trim(),Double.parseDouble(x[1].trim()),a));
    }
}
```


PS C:\Users\lenovo\Downloads\JavaProgramsss> & 'C:\Program Fi
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\Us
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'LibraryBookAutho
Effective Java,550,Joshua Bloch,jbloch@abc.com,M
Book: Effective Java
Price: 550
Author: Joshua Bloch (M), Email: jbloch@abc.com
PS C:\Users\lenovo\Downloads\JavaProgramsss>

Task 7: Sports League (Polymorphism – Method Overriding)
Problem
A sports league awards points differently depending on the sport. Cricket and Football teams follow different rules. We use **inheritance + method overriding** to handle this polymorphism.
Class Specifications:
• Team base class: name, matchesPlayed, wins, draws
• CricketTeam and FootballTeam override calculatePoints()
o Cricket: win=2, draw=1
o Football: win=3, draw=1

Solution :

```
import java.util.*;

class Team {
    private String name;
    private int matchesPlayed,wins,draws;

    Team(String n,int m,int w,int d){ name=n; matchesPlayed=m; wins=w;
draws=d; }
    String getName(){ return name; }
    int getWins(){ return wins; }
    int getDraws(){ return draws; }
    int calculatePoints(){ return 0; }
}

class CricketTeam extends Team {
    CricketTeam(String n,int m,int w,int d){ super(n,m,w,d); }
    int calculatePoints(){ return getWins()*2+getDraws(); }
    public String toString(){ return "Team: "+getName()+" (Cricket)
Points: "+calculatePoints(); }
}

class FootballTeam extends Team {
    FootballTeam(String n,int m,int w,int d){ super(n,m,w,d); }
    int calculatePoints(){ return getWins()*3+getDraws(); }
    public String toString(){ return "Team: "+getName()+" (Football)
Points: "+calculatePoints(); }
}

public class SportsLeague {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        for(int i=0;i<2;i++){
            String[] x=sc.nextLine().split(",",5);
```

```
            String n=x[1].trim();
            int m=Integer.parseInt(x[2].trim()),
w=Integer.parseInt(x[3].trim()), d=Integer.parseInt(x[4].trim());

            Team t=x[0].trim().equalsIgnoreCase("Cricket")
                ? new CricketTeam(n,m,w,d) : new FootballTeam(n,m,w,d);
            System.out.println(t);
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss>  & 'C:\Program Files\Jav
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\work
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'SportsLeague'
Cricket,India,10,6,2
Football,Barcelona,8,6,1
Team: India (Cricket) Points: 14
Team: Barcelona (Football) Points: 19
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 8: Loan Management System (Abstraction + Polymorphism)
Problem
Banks offer multiple loan types with different interest rates. Using **abstraction**, define a generic loan and calculate interest differently in subclasses.
Class Specifications:
• Abstract Loan: principal, rate, time, abstract calculateInterest()
• HomeLoan → 8%
• CarLoan → 10%
• Formula: SI = (P*R*T)/100

Solution :

```
import java.util.*;

abstract class Loan {
    private double principal,rate,time;
    Loan(double p,double r,double t){ principal=p; rate=r; time=t; }
    double getPrincipal(){ return principal; }
    double getRate(){ return rate; }
    double getTime(){ return time; }
    abstract double calculateInterest();
}

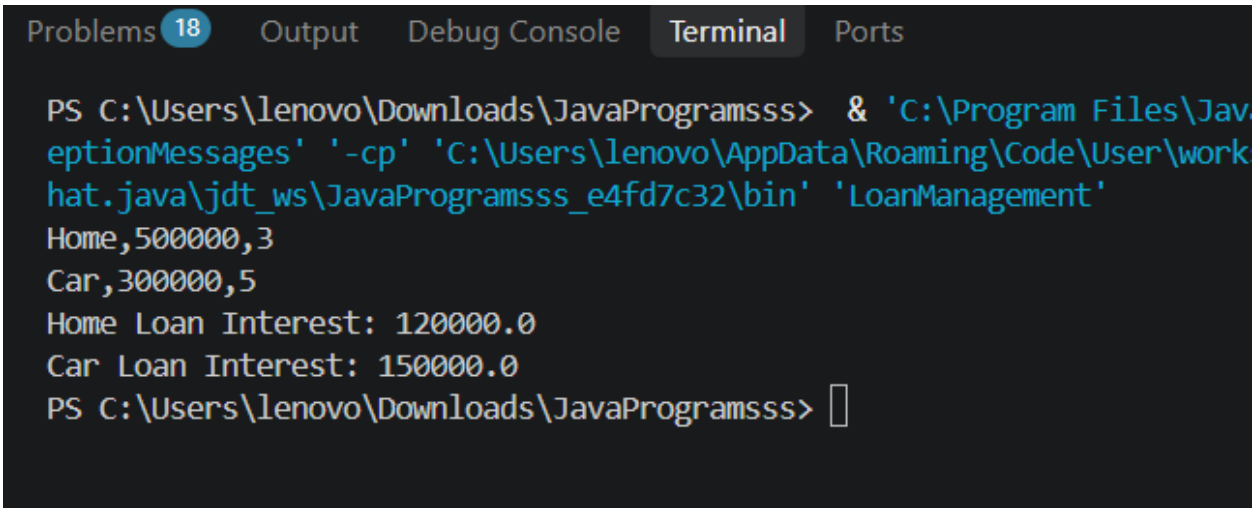
class HomeLoan extends Loan {
    HomeLoan(double p,double t){ super(p,8,t); }
    double calculateInterest(){ return
getPrincipal()*getRate()*getTime()/100; }
}

class CarLoan extends Loan {
    CarLoan(double p,double t){ super(p,10,t); }
    double calculateInterest(){ return
getPrincipal()*getRate()*getTime()/100; }
}

public class LoanManagement {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
```

```
        for(int i=0;i<2;i++){
            String[] x=sc.nextLine().split(",",3);
            String type=x[0].trim();
            double p=Double.parseDouble(x[1].trim()),
t=Double.parseDouble(x[2].trim());

            Loan l=type.equalsIgnoreCase("Home") ? new HomeLoan(p,t) : new
CarLoan(p,t);
            System.out.println(type+" Loan Interest:
"+l.calculateInterest());
        }
    }
}
```



Task 9: Online Course Platform (Association + Inheritance)
Problem
An online platform sells courses. Students can enroll in normal or premium mode. Premium students get extra discount. This uses **association + inheritance**.
Class Specifications:
• Course: courseName, duration
• Student: name, enrolledCourse (Course)
• PremiumStudent extends Student: discount

Solution :

```
import java.util.*;

class Course {
    private String courseName,duration;
    Course(String n,String d){ courseName=n; duration=d; }
    public String toString(){ return courseName+" ("+duration+")"; }
}

class Student {
    private String name;
    private Course enrolledCourse;

    Student(String n,Course c){ name=n; enrolledCourse=c; }
    String getName(){ return name; }
    Course getEnrolledCourse(){ return enrolledCourse; }

    public String toString(){ return "Student: "+name+" Course:
"+enrolledCourse; }
}

class PremiumStudent extends Student {
    private double discount;

    PremiumStudent(String n,Course c,double d){ super(n,c); discount=d; }

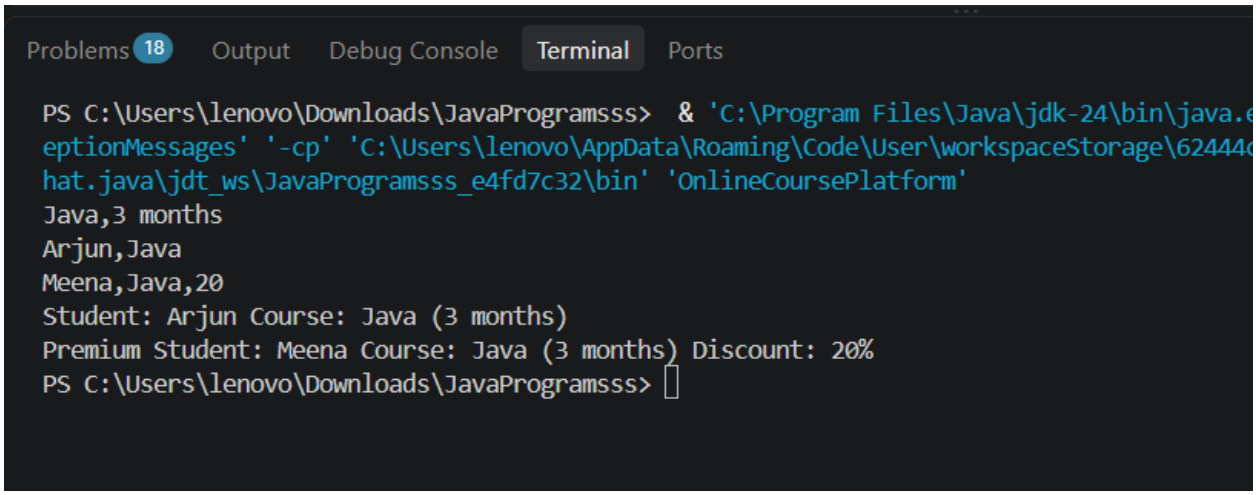
    public String toString(){
        return "Premium Student: "+getName()+" Course:
"+getEnrolledCourse()
               +" Discount: "+(long)discount+"%";
    }
}

public class OnlineCoursePlatform {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
```

```
        String[] c=sc.nextLine().split(",",2);
        Course course=new Course(c[0].trim(),c[1].trim());

        String[] s=sc.nextLine().split(",",2);
        String[] p=sc.nextLine().split(",",3);

        System.out.println(new Student(s[0].trim(),course));
        System.out.println(new
PremiumStudent(p[0].trim(),course,Double.parseDouble(p[2].trim())));
    }
}
```



Task 10: Smart Home Devices (Interfaces + Polymorphism)
Problem
Smart homes support multiple devices like fans and lights. Each device must implement standard operations (turnOn, turnOff). This demonstrates **interfaces and polymorphism**.
Class Specifications:
• Interface Device: turnOn(), turnOff()
• Fan, Light implement Device

Solution :

```
import java.util.*;

interface Device {
    void turnOn();
    void turnOff();
}

class Fan implements Device {
    public void turnOn(){ System.out.println("Fan is now ON"); }
    public void turnOff(){ System.out.println("Fan is now OFF"); }
}

class Light implements Device {
    public void turnOn(){ System.out.println("Light is now ON"); }
    public void turnOff(){ System.out.println("Light is now OFF"); }
}

public class SmartHomeDevices {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);

        for(int i=0;i<2;i++){
            Device d=sc.nextLine().trim().equalsIgnoreCase("Fan") ? new
Fan() : new Light();
            d.turnOn(); d.turnOff();
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\JavaProgramsss>  & 'C:\Program Files
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\User\
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'SmartHomeDevices'
Fan
Light
Fan is now ON
Fan is now OFF
Light is now ON
Light is now OFF
PS C:\Users\lenovo\Downloads\JavaProgramsss>
```

Task 11: University Hostel Allocation (One-to-One Relationship)
Problem
Each student in a university hostel gets exactly one room, and each room is allocated to only one student. This models a strict one-to-one relationship.
Class Specifications:
• Student: name, roll, course, room (Room)
• Room: roomNumber, block, type (Single/Double)
• Constructors + getters/setters
• toString() in Student prints room details

Solution :

```
import java.util.*;

class Room {
    private String roomNumber,block,type;

    Room() {}
    Room(String r,String b,String t){ roomNumber=r; block=b; type=t; }

    String getRoomNumber(){ return roomNumber; } void setRoomNumber(String
v){ roomNumber=v; }
    String getBlock(){ return block; } void setBlock(String v){ block=v; }
    String getType(){ return type; } void setType(String v){ type=v; }

    public String toString(){ return "Room: "+roomNumber+" "+block+"
"+type; }
}

class HostelStudent {
    private String name,roll,course;
    private Room room;

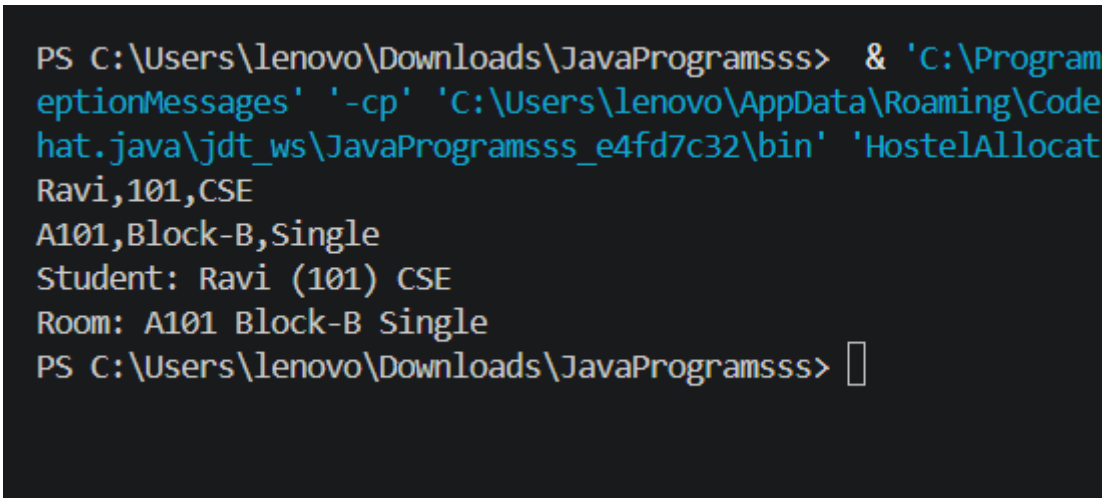
    HostelStudent() {}
    HostelStudent(String n,String r,String c,Room rm){ name=n; roll=r;
course=c; room=rm; }

    String getName(){ return name; } void setName(String v){ name=v; }
    String getRoll(){ return roll; } void setRoll(String v){ roll=v; }
    String getCourse(){ return course; } void setCourse(String v){
course=v; }
    Room getRoom(){ return room; } void setRoom(Room v){ room=v; }

    public String toString(){ return "Student: "+name+" ("+roll+")
"+course+"\n"+room; }
}
```

```
public class HostelAllocation {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] s=sc.nextLine().split(",",3),
r=sc.nextLine().split(",",3);

        Room room=new Room(r[0].trim(),r[1].trim(),r[2].trim());
        System.out.println(new
HostelStudent(s[0].trim(),s[1].trim(),s[2].trim(),room));
    }
}
```



Task 12: Vehicle Rental System (Inheritance + Overriding)
Problem
Rental companies offer cars and bikes. Both are vehicles but their rental cost calculation differs.
Class Specifications:
• Vehicle: regNo, brand, baseRate
• Car extends Vehicle: rate = baseRate * 1.5
• Bike extends Vehicle: rate = baseRate * 1.2

Solution :

```
import java.util.*;

class Vehicle {
    private String regNo,brand;
    private double baseRate;

    Vehicle(String r,String b,double rate){ regNo=r; brand=b;
baseRate=rate; }
    String getRegNo(){ return regNo; }
    String getBrand(){ return brand; }
    double getBaseRate(){ return baseRate; }
    double calculateRent(){ return baseRate; }
}

class Car extends Vehicle {
    Car(String r,String b,double rate){ super(r,b,rate); }
    double calculateRent(){ return getBaseRate()*1.5; }
    public String toString(){ return "Car "+getRegNo()+" "+getBrand()+"
Rent: "+calculateRent(); }
}

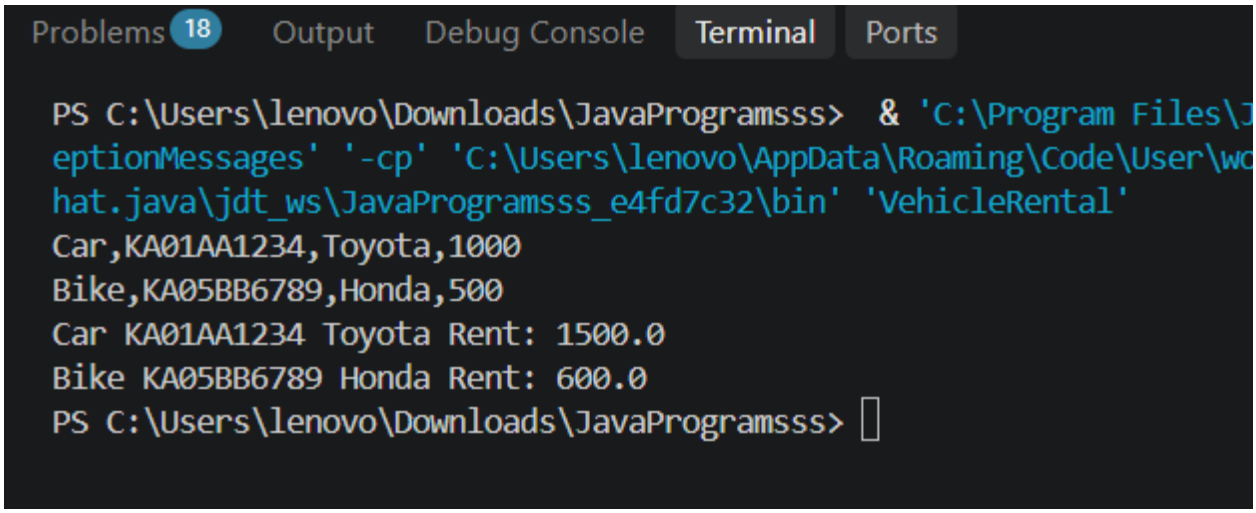
class Bike extends Vehicle {
    Bike(String r,String b,double rate){ super(r,b,rate); }
    double calculateRent(){ return getBaseRate()*1.2; }
    public String toString(){ return "Bike "+getRegNo()+" "+getBrand()+"
Rent: "+calculateRent(); }
}

public class VehicleRental {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);

        for(int i=0;i<2;i++){
            String[] x=sc.nextLine().split(",",4);
            double rate=Double.parseDouble(x[3].trim());
```

```
            Vehicle v=x[0].trim().equalsIgnoreCase("Car")
                ? new Car(x[1].trim(),x[2].trim(),rate)
                : new Bike(x[1].trim(),x[2].trim(),rate);

            System.out.println(v);
        }
    }
}
```



Task 13: Hotel Reservation System (Aggregation)
Problem
A hotel reservation includes multiple guests under a single booking.
Class Specifications:
• Guest: name, age, ID proof
• Reservation: reservationId, roomType, guests (List<Guest>)

Solution :

```
import java.util.*;

class Guest {
    private String name,idProof;
    private int age;

    Guest(String n,int a,String id){ name=n; age=a; idProof=id; }

    public String toString(){ return name+","+age+","+idProof; }
}

class Reservation {
    private String reservationId,roomType;
    private List<Guest> guests=new ArrayList<>();

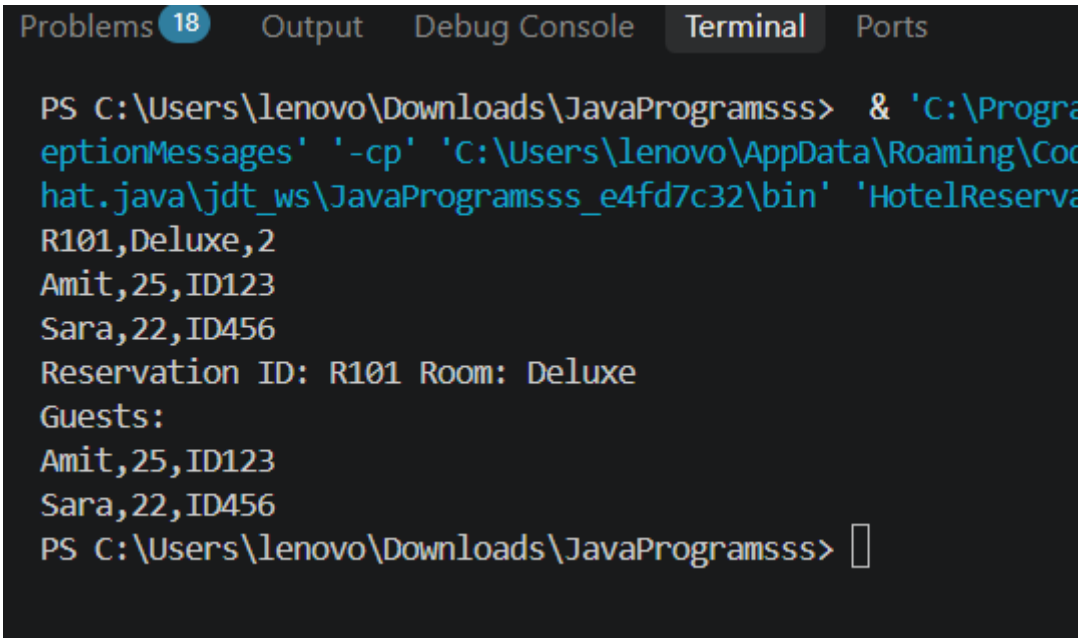
    Reservation(String id,String room){ reservationId=id; roomType=room; }
    void addGuest(Guest g){ guests.add(g); }

    public String toString(){
        String s="Reservation ID: "+reservationId+" Room:
"+roomType+"\nGuests:";
        for(Guest g:guests) s+="\n"+g;
        return s;
    }
}

public class HotelReservation {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] x=sc.nextLine().split(",",3);
        Reservation r=new Reservation(x[0].trim(),x[1].trim());

        int n=Integer.parseInt(x[2].trim());
        while(n-->0){
            String[] g=sc.nextLine().split(",",3);
            r.addGuest(new
Guest(g[0].trim(),Integer.parseInt(g[1].trim()),g[2].trim()));
```

```
            }
            System.out.println(r);
        }
}
```



Task 14: Banking ATM Simulation (Encapsulation)
Problem
Simulate an ATM where users can deposit, withdraw, and check balance. Balance must be private (encapsulation).
Class Specifications:
• Account: accNo, holderName, private balance
• Methods: deposit(), withdraw(), getBalance()

Solution :

```
import java.util.*;

class Account {
    private String accNo,holderName;
    private double balance;

    Account() {}
    Account(String a,String h,double b){ accNo=a; holderName=h; balance=b;
}

    void deposit(double a){ balance+=a; System.out.println("Deposited:
"+a); }

    void withdraw(double a){
        if(a<=balance){ balance-=a; System.out.println("Withdrawn: "+a); }
        else System.out.println("Insufficient balance");
    }

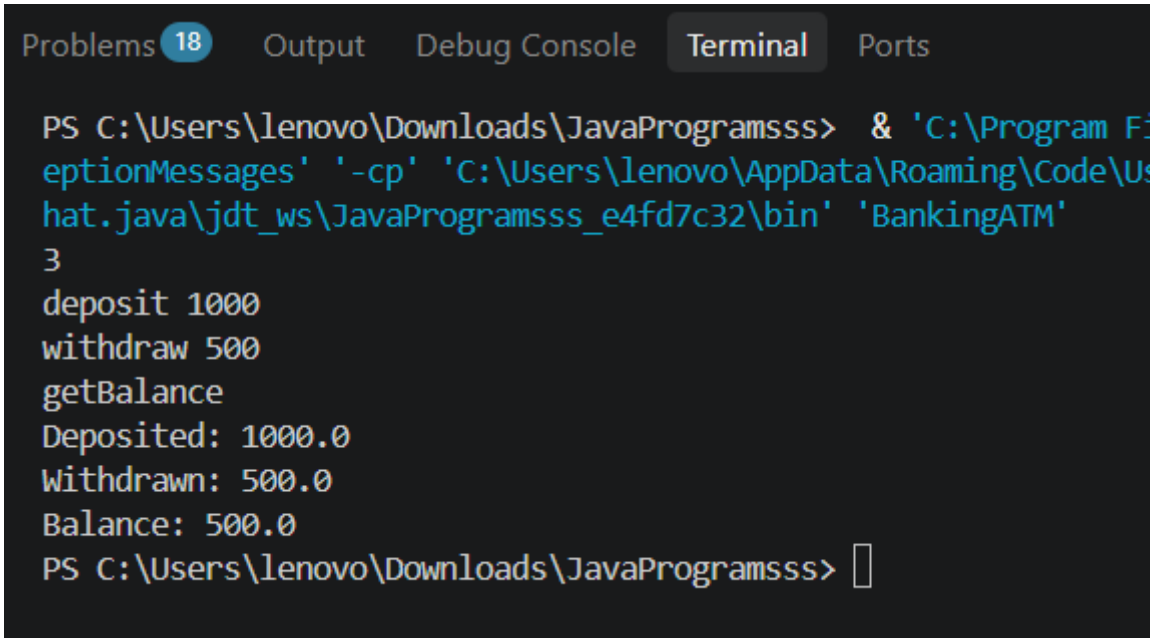
    double getBalance(){ return balance; }
}

public class BankingATM {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        Account a=new Account();

        int n=Integer.parseInt(sc.nextLine().trim());
        while(n-->0){
            String s=sc.nextLine().trim();

            if(s.startsWith("deposit"))
                a.deposit(Double.parseDouble(s.split("\\s+")[1]));
            else if(s.startsWith("withdraw"))
                a.withdraw(Double.parseDouble(s.split("\\s+")[1]));
            else
                System.out.println("Balance: "+a.getBalance());
```

```
            }
        }
}
```


Problems 18
Output
Debug Console
Terminal
Ports
PS C:\Users\lenovo\Downloads\JavaProgramsss> & 'C:\Program Fi
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\Us
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'BankingATM'
3
deposit 1000
withdraw 500
getBalance
Deposited: 1000.0
Withdrawn: 500.0
Balance: 500.0
PS C:\Users\lenovo\Downloads\JavaProgramsss>

Task 15: Passport – Citizen (One-to-One Relationship)
Problem
A passport belongs to exactly one citizen. This forms a strict one-to-one relationship.
Class Specifications:
• Citizen: name, dob, address, passport (Passport)
• Passport: passportNo, issueDate, expiryDate
• toString() in Citizen prints passport details

Solution :

```
import java.util.*;

class Passport {
    private String passportNo,issueDate,expiryDate;

    Passport(String n,String i,String e){ passportNo=n; issueDate=i;
expiryDate=e; }

    public String toString(){
        return "Passport: "+passportNo+" Issue: "+issueDate+" Expiry:
"+expiryDate;
    }
}

class Citizen {
    private String name,dob,address;
    private Passport passport;

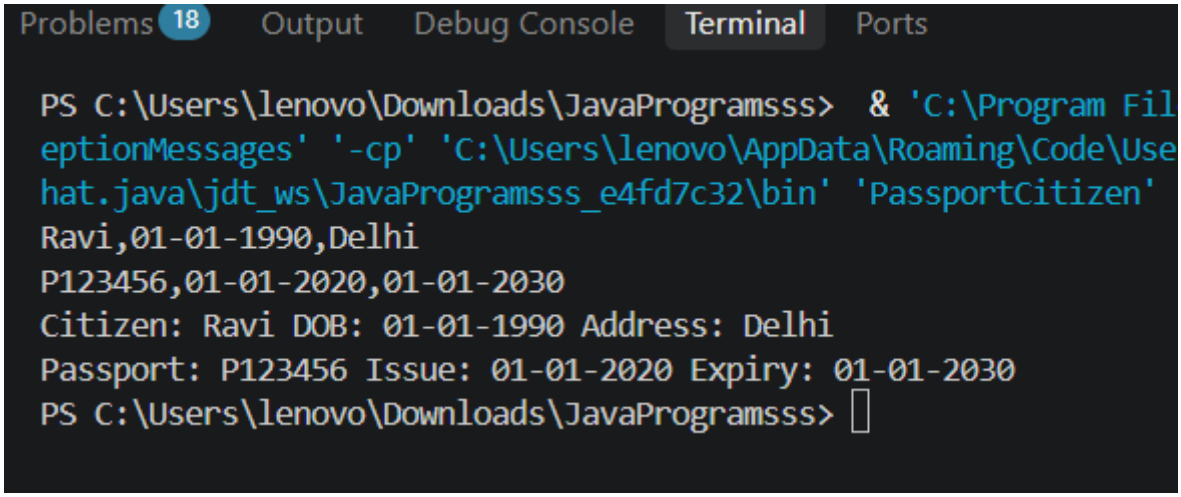
    Citizen(String n,String d,String a,Passport p){ name=n; dob=d;
address=a; passport=p; }

    public String toString(){
        return "Citizen: "+name+" DOB: "+dob+" Address:
"+address+"\n"+passport;
    }
}

public class PassportCitizen {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] c=sc.nextLine().split(",",3),
p=sc.nextLine().split(",",3);

        Passport pass=new Passport(p[0].trim(),p[1].trim(),p[2].trim());
        System.out.println(new
Citizen(c[0].trim(),c[1].trim(),c[2].trim(),pass));
```

```
    }
}
```


Problems 18
Output
Debug Console
Terminal
Ports
PS C:\Users\lenovo\Downloads\JavaProgramsss> & 'C:\Program Fil
eptionMessages' '-cp' 'C:\Users\lenovo\AppData\Roaming\Code\Use
hat.java\jdt_ws\JavaProgramsss_e4fd7c32\bin' 'PassportCitizen'
Ravi,01-01-1990,Delhi
P123456,01-01-2020,01-01-2030
Citizen: Ravi DOB: 01-01-1990 Address: Delhi
Passport: P123456 Issue: 01-01-2020 Expiry: 01-01-2030
PS C:\Users\lenovo\Downloads\JavaProgramsss>